



SmartPot

DOCUMENTACIÓN DEL COMPONENTE

SmartPot-API

El centro de la plataforma SmartPot

REST con JWT, puente MQTT con el broker, cultivos reales y virtuales, agente de automatización con el asistente de IA, canales de notificación, seguridad, configuración y operación de la API.



DOCUMENTO
SmartPot-API

VERSIÓN
1.1 · octubre 2026

EQUIPO
SmartPotTech

PROYECTO
smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Componente	SmartPot-API
Versión	1.1 · octubre 2026
Alcance	Dominios de la API, cultivos reales y virtuales, flujo de lecturas y comandos, seguridad, configuración, pruebas y operación
Documentación de la plataforma	Documentación técnica , recorrido del proyecto , ciclo de vida y diagramas generales
Mantenimiento	Se genera desde docs/ de este repositorio con las herramientas de .github/docs/tools ; se actualiza con cada cambio del componente

CONTENIDO

Contenido

1. Propósito
2. Arquitectura del componente
3. Cultivos reales y virtuales
4. De la lectura a la orden
5. Seguridad de cada petición
6. Contrato
7. Configuración
8. Pruebas
9. Operación

PARTE I

El componente

SECCIÓN 01

Propósito

En palabras simples

La API es la única puerta de SmartPot. La PWA le habla por HTTPS; los dispositivos de los cultivos, por MQTT a través del broker. La API guarda las lecturas, le pregunta al asistente de IA cómo va cada cultivo, manda las órdenes a los actuadores, administra las cuentas MQTT de cada cultivo y avisa por la PWA y por Telegram. Todo lo que ve una persona pasa por aquí, siempre con su sesión y solo sobre sus propios cultivos.

RESPONSABILIDAD	CÓMO
Cuentas y sesión	Registro, ingreso con JWT, recuperación de contraseña por correo, perfil y borrado de la cuenta
Cultivos	Reales o virtuales (se elige al crear y no cambia), seis especies, cuatro formas, su lugar con el clima de afuera y modo automático
Dispositivos	Cuenta MQTT por cultivo con clave de 192 bits cifrada con AES-256-GCM; aprovisionamiento en Mosquitto
Lecturas	Telemetría MQTT validada contra el rango físico de cada sensor, historial, resumen y CSV
Órdenes	Actuadores como switches (409 si la orden no cambia nada o se cruza con otra), comandos con QoS 1, confirmación del dispositivo, vencimiento y órdenes en bloque
Asistente	Evaluación con SmartPot-AI, agente de automatización y envío de lecturas reales para el aprendizaje
Simulación	Configuración de los cultivos virtuales y proxy al simulador interno
Avisos	Notificaciones en la PWA y en Telegram, con canales intercambiables, avisos por cultivo, resúmenes y chats compartidos



SECCIÓN 02

Arquitectura del componente

Cada dominio sigue las mismas capas (controller, service, repository, mapper y model). Los dominios se hablan con servicios y eventos de Spring (`ReadingRecordedEvent`, `CropDeletedEvent`, `DeviceKeyRotatedEvent`, `NotificationCreatedEvent`), así un cambio en uno no arrastra a los demás.

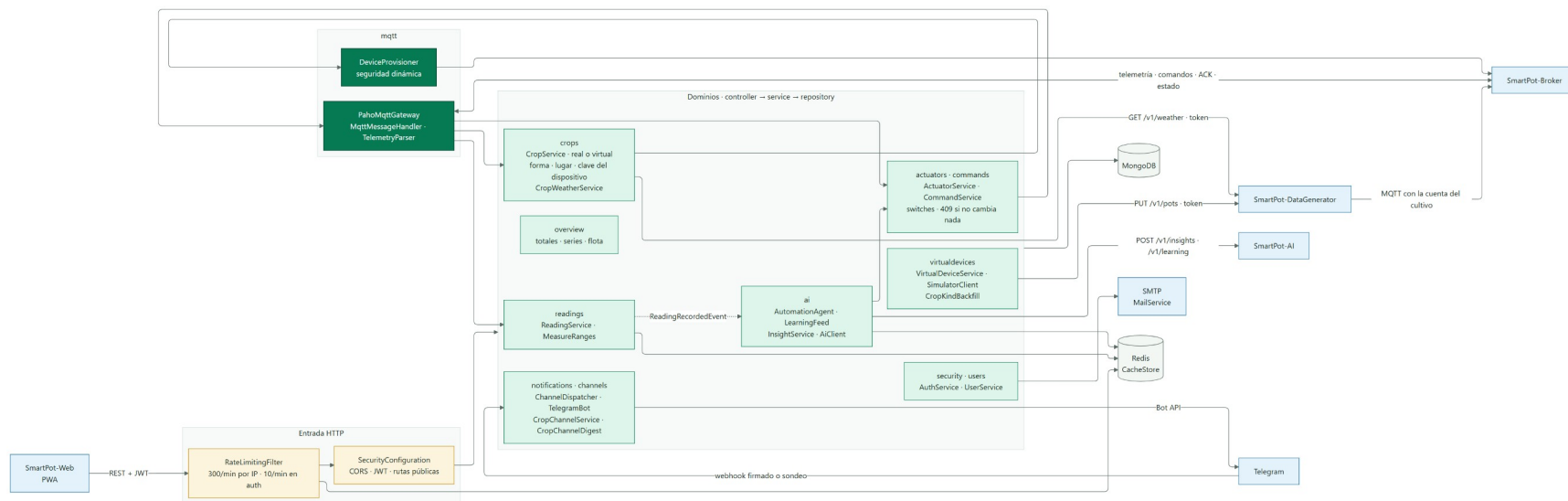


Figura 1 · SmartPot-API por dentro



MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}, "layout": "elk", "elk": {"nodePlacementStrategy": "BRANDES_KOEPF", "mergeEdges":
false, "cycleBreakingStrategy": "GREEDY"}}}%
flowchart LR
    pwa["SmartPot-Web<br/>PWA"]
    tg["Telegram"]
    subgraph borde["Entrada HTTP"]
        direction TB
        sec["SecurityConfiguration<br/>CORS · JWT · rutas públicas"]
        rate["RateLimitingFilter<br/>300/min por IP · 10/min en auth"]
    end
    end
    subgraph dominio["Dominios · controller → service → repository"]
        direction TB
        auth["security · users<br/>AuthService · UserService"]
        crops["crops<br/>CropService · real o virtual<br/>forma · lugar · clave del
dispositivo<br/>CropWeatherService"]
        readings["readings<br/>ReadingsService · MeasureRanges"]
        actuators["actuators · commands<br/>ActuatorService · CommandService<br/>switches · 409
si no cambia nada"]
        overview["overview<br/>totales · series · flota"]
        notif["notifications · channels<br/>ChannelDispatcher ·
TelegramBot<br/>CropChannelService · CropChannelDigest"]
        virtual["virtualdevices<br/>VirtualDeviceService · SimulatorClient<br/>CropKindBackfill"]
        ai["ai<br/>AutomationAgent · LearningFeed<br/>InsightService · AiClient"]
    end
    end
    subgraph mqtt["mqtt"]
        direction TB
        gateway["PahoMqttGateway<br/>MqttMessageHandler · TelemetryParser"]
        prov["DeviceProvisioner<br/>seguridad dinámica"]
    end
    end
    mongo[("MongoDB")]
    redis[("Redis<br/>CacheStore")]
    broker["SmartPot-Broker"]
    aiscv["SmartPot-AI"]
    sim["SmartPot-DataGenerator"]
    smtp["SMTP<br/>MailService"]
    pwa →|"REST + JWT"| rate → sec → dominio
    tg →|"webhook firmado o sondeo"| notif
    notif →|"Bot API"| tg
    dominio → mongo
    rate & readings & ai → redis
    auth → smtp
    crops → prov → broker
    gateway <→|"telemetría · comandos · ACK · estado"| broker
```



```
gateway → readings & actuators & crops
readings -.→|"ReadingRecordedEvent"| ai
ai →|"POST /v1/insights · /v1/learning"| aiscv
ai → actuators → gateway
virtual →|"PUT /v1/pots · token"| sim
crops →|"GET /v1/weather · token"| sim
sim →|"MQTT con la cuenta del cultivo"| broker
classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
class pwa,tg,broker,aisvc,sim,smtp water
class sec,rate sun
class auth,crops,readings,actuators,overview,notif,virtual,ai leaf
class gateway,prov core
class mongo,redis muted
```

PAQUETE	QUÉ CONTIENE
security, users	Filtros, JWT, límite de peticiones, AES-GCM, autenticación y perfil
crops	Cultivos, tipo y forma, modo automático y credenciales del dispositivo
readings	Lecturas, rangos físicos, resumen y exportación
actuators, commands	Actuadores de cada cultivo y ciclo de vida de las órdenes
mqtt	Cliente Paho, tópicos v1, parser de telemetría y aprovisionamiento en el broker
ai	Cliente de la IA, evaluación, agente de automatización y LearningFeed
overview	Panel general: totales, series con <code>\$dateTrunc</code> y análisis de flota
notifications, channels	Alertas y canales externos (<code>NotificationChannel</code> , hoy Telegram)
virtualdevices	Simulación de los cultivos virtuales, reconciliación y completado de cultivos anteriores
cache, mail, health, config	Redis con respaldo en memoria, correo, <code>/health</code> y configuración

SECCIÓN 03

Cultivos reales y virtuales

Al crear un cultivo se elige, una sola vez, de dónde vienen sus lecturas. Los dos tipos tienen cuenta en el broker; lo que cambia es quién la usa.

REGLA	REAL	VIRTUAL
Quién publica	Un ESP32 con el firmware, físico o simulado en Wokwi	SmartPot-DataGenerator
Credenciales	Se entregan una vez y se pueden rotar	No se entregan: la API se las pasa al simulador por la red interna
Actuadores al nacer	Bomba, luz ultravioleta y ventilador	Los seis
Rutas propias	/device y /device/key	/virtual-device (PUT cambia o reanuda, DELETE pausa)
Aprendizaje continuo	Sus lecturas entrenan los modelos	Sus lecturas no se envían
Límite	20 cultivos por cuenta en total	Hasta 5 por cuenta

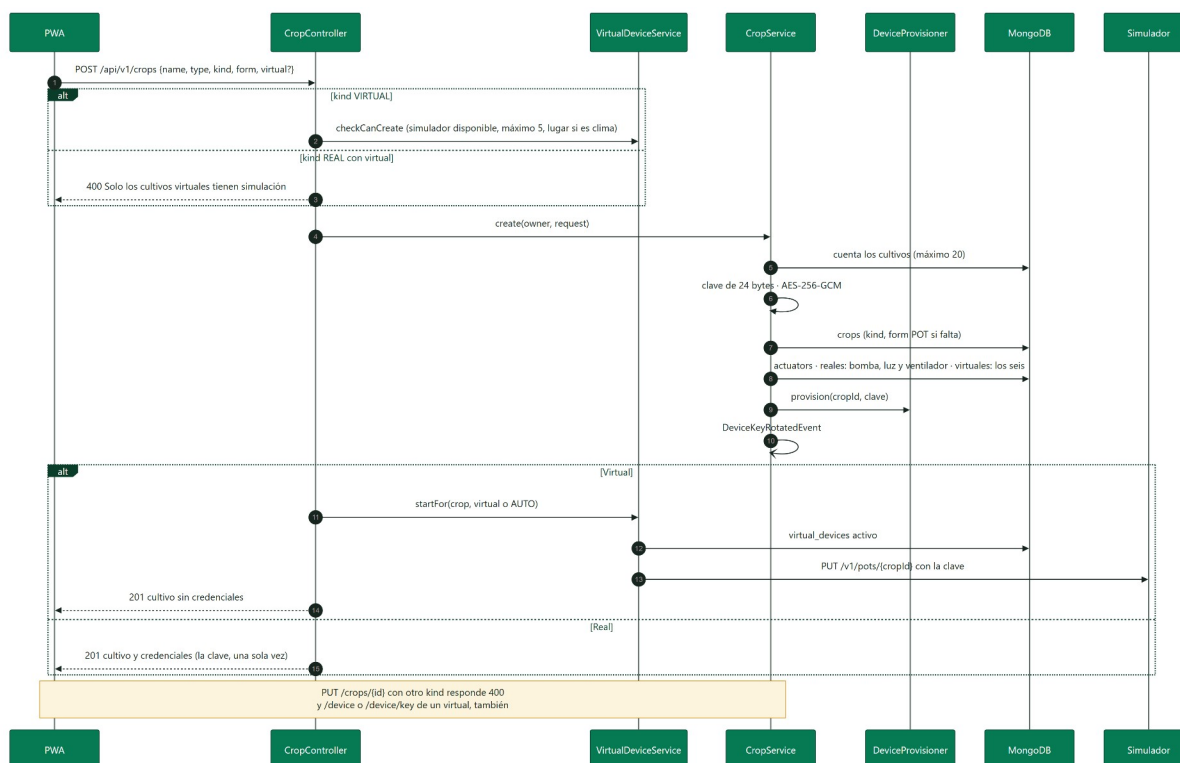


Figura 2 · Crear un cultivo real o virtual



```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    participant W as PWA
    participant C as CropController
    participant V as VirtualDeviceService
    participant S as CropService
    participant P as DeviceProvisioner
    participant D as MongoDB
    participant SIM as Simulador
    W->>C: POST /api/v1/crops {name, type, kind, form, virtual?}
    alt kind VIRTUAL
        C->>V: checkCanCreate (simulador disponible, máximo 5, lugar si es clima)
    else kind REAL con virtual
        C-->>W: 400 Solo los cultivos virtuales tienen simulación
    end
    C->>S: create(owner, request)
    S->>D: cuenta los cultivos (máximo 20)
    S->>S: clave de 24 bytes · AES-256-GCM
    S->>D: crops (kind, form POT si falta)
    S->>D: actuators · reales: bomba, luz y ventilador · virtuales: los seis
    S->>P: provision(cropId, clave)
    S->>S: DeviceKeyRotatedEvent
    alt Virtual
        C->>V: startFor(crop, virtual o AUTO)
        V->>D: virtual_devices activo
        V->>SIM: PUT /v1/pots/{cropId} con la clave
        C-->>W: 201 cultivo sin credenciales
    else Real
        C-->>W: 201 cultivo y credenciales (la clave, una sola vez)
    end
    Note over W,S: PUT /crops/{id} con otro kind responde 400<br/>y /device o /device/key de un
    virtual, también

```

La forma (**POT**, **NFT**, **TOWER**, **RAFT**) solo decide cómo lo dibuja la PWA y se puede editar. Los cultivos creados antes de existir el tipo se completan al arrancar con **CropKindBackfill**: los que tenían simulación pasan a virtuales, el resto a reales, y sin forma quedan como maceta.



PARTE II

Funcionamiento

SECCIÓN 04

De la lectura a la orden

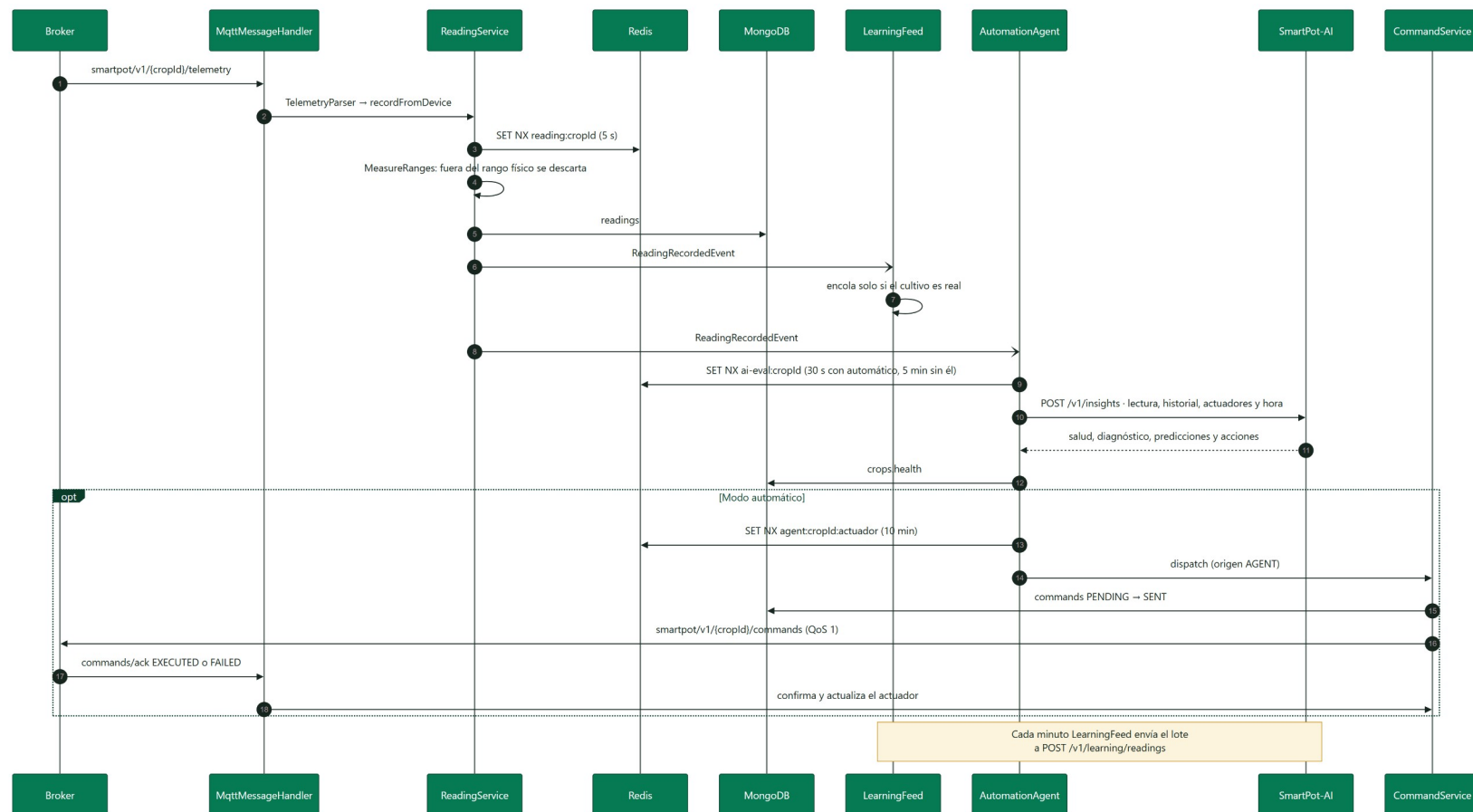


Figura 3 · De la lectura a la orden automática



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
sequenceDiagram
    autonumber
    participant B as Broker
    participant H as MqttMessageHandler
    participant R as ReadingService
    participant K as Redis
    participant D as MongoDB
    participant L as LearningFeed
    participant G as AutomationAgent
    participant I as SmartPot-AI
    participant M as CommandService
    B->>H: smartpot/v1/{cropId}/telemetry
    H->>R: TelemetryParser → recordFromDevice
    R->>K: SET NX reading:cropId (5 s)
    R->>R: MeasureRanges: fuera del rango físico se descarta
    R->>D: readings
    R->>L: ReadingRecordedEvent
    L->>L: encola solo si el cultivo es real
    R->>G: ReadingRecordedEvent
    G->>K: SET NX ai-eval:cropId (30 s con automático, 5 min sin él)
    G->>I: POST /v1/insights · lectura, historial, actuadores y hora
    I-->>G: salud, diagnóstico, predicciones y acciones
    G->>D: crops.health
    opt Modo automático
        G->>K: SET NX agent:cropId:actuador (10 min)
        G->>M: dispatch (origen AGENT)
        M->>D: commands PENDING → SENT
        M->>B: smartpot/v1/{cropId}/commands (QoS 1)
        B->>H: commands/ack EXECUTED o FAILED
        H->>M: confirma y actualiza el actuador
    end
    Note over L,I: Cada minuto LearningFeed envía el lote a POST /v1/learning/readings
```

REGLA	VALOR
Lecturas por cultivo	Máximo una cada 5 s; las demás se descartan
Validación	Cada variable dentro del rango físico del sensor; si no, se descarta la lectura completa



REGLA	VALOR
Evaluación	Como máximo cada 30 s con modo automático y cada 5 min sin él
Enfriamiento del agente	10 minutos por actuador
Vencimiento de un comando	2 minutos sin ACK → EXPIRED y aviso
Lote de aprendizaje	Cada minuto, hasta 1000 lecturas por envío; la cola guarda 20 000

SECCIÓN 05

Seguridad de cada petición

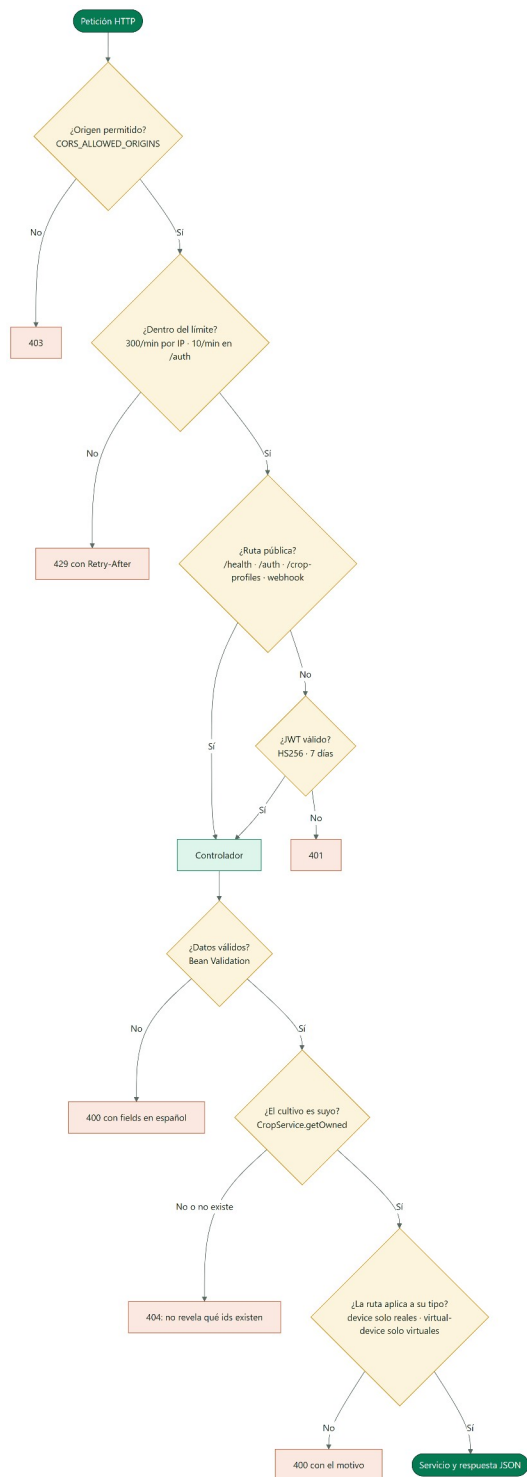


Figura 4 · Qué revisa la API en cada petición



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    req(["Petición HTTP"]) --> cors["cors{¿Origen permitido?<br/>CORS_ALLOWED_ORIGINS}"]
    cors -->|No| r403["403"]
    cors -->|Sí| rate["rate{¿Dentro del límite?<br/>300/min por IP · 10/min en /auth}"]
    rate -->|No| r429["429 con Retry-After"]
    rate -->|Sí| pub["pub{¿Ruta pública?<br/>/health · /auth · /crop-profiles · webhook}"]
    pub -->|Sí| ctrl["Controlador"]
    pub -->|No| jwt["jwt{¿JWT válido?<br/>HS256 · 7 días}"]
    jwt -->|No| r401["401"]
    jwt -->|Sí| ctrl
    ctrl --> valid["valid{¿Datos válidos?<br/>Bean Validation}"]
    valid -->|No| r400["400 con fields en español"]
    valid -->|Sí| owner["owner{¿El cultivo es suyo?<br/>CropService.getOwned}"]
    owner -->|No o no existe| r404["404: no revela qué ids existen"]
    owner -->|Sí| kind["kind{¿La ruta aplica a su tipo?<br/>device solo reales · virtual-device solo virtuales}"]
    kind -->|No| r400b["400 con el motivo"]
    kind -->|Sí| ok(["Servicio y respuesta JSON"])
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class req,ok core
    class cors,rate,pub,jwt,valid,owner,kind sun
    class r403,r429,r401,r400,r404,r400b clay
    class ctrl leaf
```

CONTROL	DETALLE
Contraseñas	8 caracteres a 72 bytes con mayúscula, minúscula y número; BCrypt de costo 12
Recuperación	Responde 202 aunque el correo no exista; token de 30 minutos guardado como SHA-256
Claves de los dispositivos	24 bytes aleatorios, cifrados con AES-256-GCM; la llave es SMARTPOT_AES_KEY
Servicios internos	La IA y el simulador se llaman con token de servicio por la red interna



CONTROL	DETALLE
Telegram	Webhook con secreto comparado en tiempo constante; códigos de vinculación de un solo uso

SECCIÓN 06

Contrato

La documentación interactiva vive en [/docs](#) y el esquema en [/v3/api-docs](#); el [README](#) resume todas las rutas y la [documentación técnica](#) detalla el contrato MQTT v1.

GRUPO	RUTAS
Público	/health, /api/v1/auth/*, /api/v1/crop-profiles
Cultivos	/api/v1/crops, /crops/{id}, /crops/automation, /crops/{id}/device, /crops/{id}/weather
Datos del cultivo	/readings, /actuators, /commands, /insights, /virtual-device, /channels
Cuenta	/api/v1/users/me, /notifications, /channels, /overview, /commands, /ai/learning



PARTE III

Operación

SECCIÓN 07

Configuración

La API lee `.env` desde la carpeta del proyecto; `.env.example` lista todas las variables.

GRUPO	VARIABLES
Servidor	PORT, LOG_LEVEL, WEB_BASE_URL, PUBLIC_API_URL, CORS_ALLOWED_ORIGINS, API_DOCS_ENABLED
Secretos	SMARTPOT_JWT_SECRET (32 caracteres o más), SMARTPOT_AES_KEY (AES-256 en Base64), JWT_EXPIRATION
Datos	MONGODB_URI, REDIS_HOST, REDIS_PORT, REDIS_PASSWORD
Correo	MAIL_ENABLED, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM
MQTT	MQTT_ENABLED, MQTT_BROKER_URI, MQTT_ADMIN_USERNAME, MQTT_ADMIN_PASSWORD, MQTT_PUBLIC_*, MQTT_WEBSOCKET_URL
IA	AI_ENABLED, AI_BASE_URL, SMARTPOT_AI_TOKEN, AI_EVALUATION_INTERVAL, AI_AUTOMATION_COOLDOWN, AI_LEARNING_*
Telegram	TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_USERNAME, TELEGRAM_MODE (polling o webhook), TELEGRAM_WEBHOOK_SECRET
Simulador	SIMULATOR_ENABLED, SIMULATOR_BASE_URL, SIMULATOR_TOKEN (sin él no se crean cultivos virtuales)

LLAVE AES

Si SMARTPOT_AES_KEY se pierde o cambia, las cuentas MQTT no se pueden reconstruir: cada dueño debe rotar la clave de sus cultivos reales.

SECCIÓN 08

Pruebas

`./mvnw verify` corre 147 pruebas con JUnit y Mockito: cifrado, JWT, contraseñas, tópicos y telemetría MQTT, aprovisionamiento, comandos y switches, cultivos con su tipo fijo, su forma, su lugar y su clima, los avisos por cultivo, el agente, el envío de lecturas para el aprendizaje (solo reales), canales y bot de Telegram, la simulación de los virtuales (dueño, clave, límites, pausa y reconciliación), el completado de



cultivos anteriores, la caché con respaldo local y la cadena de seguridad de los controladores. El E2E central de [.github](#) recorre la API completa con 46 comprobaciones.

SECCIÓN 09

Operación

TAREA	CÓMO
Estado	GET /health → base, broker, caché, IA y simulador (503 si la base cae)
Logs	docker logs -f smartpot-api
Imagen	ghcr.io/smartpottech/smartpot-api: compila en una etapa aparte, usuario 1000, solo lectura con tmpfs en /tmp y HEALTHCHECK
Despliegue	Cada cambio en main pasa por el CI, publica la imagen y pide el despliegue central de .github
Rotar una clave	Pestaña Dispositivo de la PWA o POST /api/v1/crops/{id}/device/key (solo reales)